

April 2026 - 6

Point-in-Time Recovery and Audit Logging



Figure 1: Figure 0: Point-in-Time Recovery and Audit Logging

Welcome to this edition of yugabyteDB Developer's Notebook (YDN). This month we answer the following question(s);

Two separate but related asks from our compliance team: first, we need a fast way to undo a bad deploy or an accidental DELETE without restoring a full backup from last night. Second, we need an audit trail of who ran what against a specific sensitive table. Does yugabyteDB have real, native answers for both, and what are the honest limitations ?

Yes to both, and both have real, specific edges worth knowing before you rely on them. Point-in-Time Recovery (PITR) is yugabyteDB's fast "oops button" – distinct from a full backup, and built on a genuinely clever hard-link mechanism rather than copying data. Audit logging is provided by pgAudit, configurable down to the specific role and table – but it has several real, confirmed gotchas around exactly which configuration commands actually take effect. Both are covered this month with real, verified behavior, not just the documented happy path.

Software versions The primary software components used in this edition of YDN include yugabyteDB Anywhere (YBA) and yugabyteDB (YB) version 2025.2.0.0-b131. All of the steps below are run on one very large sized Mac Book Pro, or if you prefer, run these steps on yugabyteDB Aeon, yugabyteDB's managed service, or Amazon Web Services (AWS), Google Cloud Platform (GCP), or another hyper-scaler.

For isolation and (simplicity), we develop and test all systems inside virtual machines or emulated machines, using the desktop hypervisors VMware Fusion version 13.6.4, and/or UTM version 4.6.5. Generally we run a single node of YBA, and 8 nodes of YB (5 nodes in one region, and 3 nodes in a second region) using Alma Linux versions 8 and 9, and Ubuntu Desktop version 24.04. We also run a client node using Ubuntu Desktop version 24.04.

6.1 Terms and core concepts

6.1.1 Introduction: Backups and PITR Solve Different Problems

Backups and Point-in-Time Recovery (PITR) both protect data, but answer genuinely different questions and should not be thought of as one feature with two names. Backups exist for long-term protection, migration, and true disaster recovery – they can restore to another cluster, restore selected tables in some cases, and survive total cluster loss. PITR exists specifically for fast rollback from a *recent* mistake – a bad DELETE, a

bad deploy – and rewinds a database or keyspace to an earlier point using in-cluster distributed snapshots, not a restore from external storage. The honest framing, worth keeping in mind before reaching for either: **PITR cannot protect you from total cluster loss** (it’s an in-cluster mechanism), and **backups cannot restore you to “10 seconds before the mistake”** (they’re only ever as fresh as the last completed backup) – and a PITR restore rewinds history, meaning everything written after the chosen restore point is genuinely lost, not merely hidden.

6.1.2 How a Snapshot Actually Works: Hard Links, Not Copies

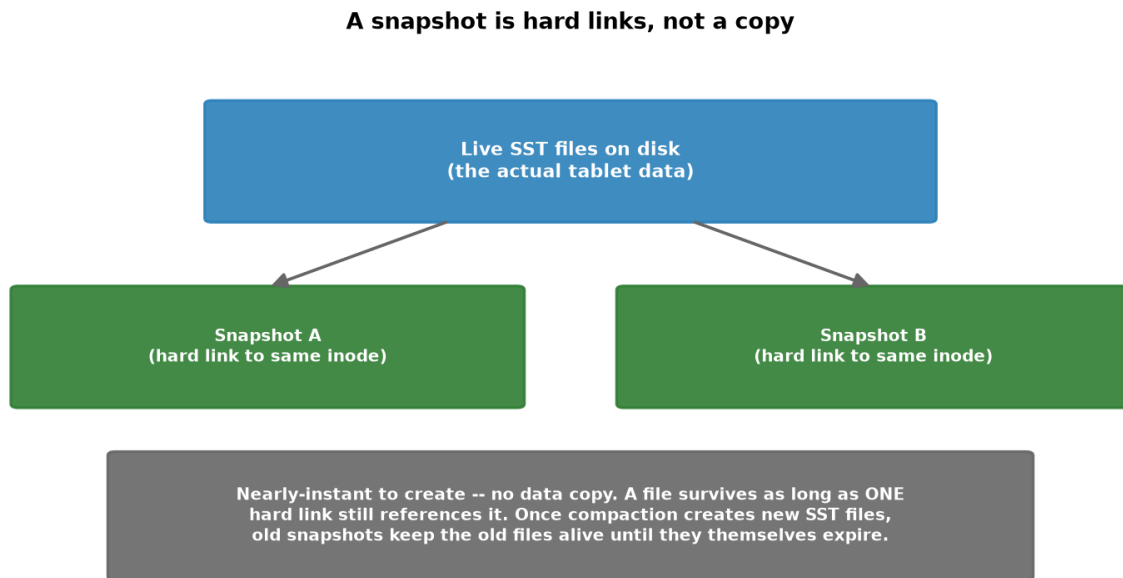


Figure 2: Figure 1: A snapshot is hard links, not a copy

When yugabyteDB creates a snapshot, it does not copy the underlying SST data files – it creates **hard links** to the existing files, in per-tablet snapshot directories living on the same storage volumes as the live data. A hard link points directly at the same underlying inode as the original file; the file’s actual content is only removed from disk once *every* hard link referencing it (the live data’s own reference, plus any snapshot’s reference) has been removed. This is precisely what makes snapshot creation and restoration fast – there’s no bulk data copy happening at snapshot-creation time at all.

The corollary is a real, worth-knowing storage-growth mechanic: as the live database keeps evolving and background compactions produce new SST files, older snapshots continue referencing the *old* files via their hard links, keeping those old files alive on disk even after the live data has moved on. Keeping many snapshots, or snapshots with long retention windows, directly increases how much old, otherwise-compactable data stays resident on disk. Retention policy is therefore not just a compliance/RPO decision – it’s directly a storage-capacity decision.

Example 6-1: Creating and tuning a PITR snapshot schedule

```

yb-admin create_snapshot_schedule <snapshot-interval-minutes> <retention-minutes> <filter-expression>
yb-admin edit_snapshot_schedule <schedule-id> [interval <minutes>] [retention <minutes>]
  
```

Shorter intervals and longer retention both increase how precisely and how far back you can restore, at the direct cost of more retained (hard-linked) snapshot data. A related gflag governs how much concurrent snapshot activity the cluster allows: `--max_concurrent_snapshot_rpcs` (YB-Master) directly

caps total concurrent tablet snapshot RPCs cluster-wide when set to a value ≥ 0 ; when left negative, `--max_concurrent_snapshot_rpcs_per_tserver` governs the same thing on a per-node basis instead.

6.1.3 Real Limitations, Confirmed Against the Documentation and the Cluster

PITR operates at the database or keyspace level, and explicitly does **not** restore global objects – tablespaces, roles, and permissions are outside its scope, since those are cluster-wide rather than per-database. PITR also cannot be used to recover from a YSQL system catalog upgrade; a full snapshot/backup restore is the correct recovery path for that specific case. Comparing the two mechanisms directly on the dimensions that matter most in an incident:

Capability	Backups	PITR
Table-level restore	Limited	No (database/keyspace-level only)
Cross-cluster restore	Yes	No
Restore to “N seconds ago”	No (last completed backup only)	Yes
Survives total cluster loss	Yes	No (in-cluster mechanism)

A real, working technique for effectively restoring “just one table” despite PITR itself operating at the database level: clone the entire database as of a specific historical timestamp, then pull the one table you actually need out of the clone.

Example 6-2: Cloning a database as of a PITR timestamp

```
-- Get a human-readable current timestamp for reference:
SELECT to_char(clock_timestamp(), 'YYYY-MM-DD HH24:MI:SS.US TZ') AS wall_clock_time;

-- Clone the whole database as it existed at a specific past moment:
CREATE DATABASE my_dbnw_copy TEMPLATE my_dbnw AS OF '2026-01-14 14:12:00.000000';
```

This requires `enable_db_clone = true` (a YB-Master gflag) to be set ahead of time. The clone itself relies on the same PITR/snapshot machinery under the hood, just pointed at creating a new, separate database rather than rewinding the existing one – a genuinely useful pattern when the actual need is “get back this one row/table” rather than “undo everything since this timestamp.”

6.1.4 Audit Logging: pgAudit, and What Actually Takes Effect

pgAudit provides fine-grained SQL audit logging, configurable at the session level or the object (table) level – object-level scoping is YSQL-only. Enabling it through YBA requires Enhanced Postgres Compatibility considerations: with that setting on, `ysql_pg_conf_csv` (and specifically `log_line_prefix` within it) cannot be set manually, and audit logging is instead enabled through YBA’s own universe-level configuration path (Admin -> Advanced -> Global Config -> `yb.universe.audit_logging_enabled`, followed by enabling logging under the specific universe’s own Logs tab, and a rolling restart to apply it).

Example 6-3: The real pgAudit settings this project verified

```
pgaudit.log           = 'all'      -- DMLs, DDLs, ROLE, FUNCTION, etc.
pgaudit.log_catalog    = off        -- don't log system catalog access
pgaudit.log_parameter = on         -- show actual bound values, not just placeholders
pgaudit.log_relation   = on         -- show which table(s)/index(es) a statement touched
suppress_nonpg_logs    = on         -- suppress RPC/tablet-server noise from the log file
```

6.1.5 Real, Confirmed Configuration Gotchas

Several specific configuration paths were tried and directly verified to *not* behave as their syntax alone would suggest:

pgAudit: what actually works, confirmed live



Figure 3: Figure 2: pgAudit – what actually works, confirmed live

- **SET pgaudit.role = '...'** at the session level: **no error, but no effect whatsoever.** The setting is silently accepted and silently does nothing. The commands that actually work are **ALTER ROLE ALL SET pgaudit.role = 'my_auditor';** or **ALTER DATABASE my_dbnw SET pgaudit.role = 'my_auditor';** – and even these require reconnecting the session before the new setting takes effect; it does not apply retroactively to an already-open connection.
- **ALTER SYSTEM SET pgaudit.log = '...'** and **ALTER SYSTEM SET shared_preload_libraries = '...': both return “Not supported yet.”** Configuration of pgAudit and its preloaded libraries goes through YBA’s own universe-level gflag/config mechanism (section 6.1.4), not PostgreSQL’s own **ALTER SYSTEM** mechanism.
- **Object-level audit logging covers SELECT, UPDATE, INSERT, DELETE – explicitly not TRUNCATE.** Worth knowing directly if your compliance requirement includes catching a table being truncated, since pgAudit’s object-level scope will not surface that specific operation the way it does the four DML statement types it does cover.
- **Object-level scoping is controlled by ordinary object privileges, not a separate audit-specific grant mechanism.** Once pgaudit.role is set to an audit role (e.g., my_auditor), **GRANT SELECT ON t1 TO my_auditor;** causes **SELECT**s against t1 to be logged; **REVOKE SELECT ON t1 FROM my_auditor;** turns that logging back off for that table. This is a clean, reuses-what-already-exists design – but it also means anyone who can **GRANT/REVOKE** privileges on a table can silently change what gets audited on it, which is worth factoring into who holds those grant privileges in the first place.

[!NOTE] None of these behaviors are bugs – each is either a deliberate design choice (routing configuration through YBA rather than **ALTER SYSTEM**) or a real, current platform limitation (**TRUNCATE** not covered at the object level). The value here is simply having them confirmed directly, rather than assuming a command that runs without an error also did what its syntax implies.

6.2 Complete the following

In this section we build and verify both halves of this article directly: a PITR snapshot schedule with a real single-table recovery via database cloning, and pgAudit configured correctly the first time, using the exact commands confirmed to actually take effect in section 6.1.5.

6.2.1 Prerequisites

A yugabyteDB cluster with YBA, yb-admin access, and permission to edit universe gflags (`enable_db_clone`, and YBA's `yb.universe.audit_logging_enabled` global config setting).

6.2.2 Step 1 – Enable a PITR snapshot schedule

Via YBA: navigate to the target universe's **Backups -> PITR -> Enable**, choose YSQL or YCQL and the target database, and set a retention period (7 days default, 2-day minimum). Confirm the schedule is active, then note the current wall-clock time via `SELECT to_char(clock_timestamp(), 'YYYY-MM-DD HH24:MI:SS.US TZ') AS wall_clock_time`; before making a test change.

6.2.3 Step 2 – Make a destructive change, then recover just one table

Make a deliberate, reversible-for-testing-purposes change (e.g., `DELETE FROM us_states`; against a disposable test table), then set `enable_db_clone = true` (YB-Master gflag, via **Edit Flags -> Add to Master**) and reproduce Example 6-2's clone-based single-table recovery:

```
CREATE DATABASE my_dbnw_copy TEMPLATE my_dbnw AS OF '<timestamp from Step 1>';
```

Confirm the cloned database still has the deleted rows, and pull them back into the live database from the clone.

6.2.4 Step 3 – Configure pgAudit correctly, using only the commands confirmed to work

Example 6-4: The correct sequence, skipping the two documented dead ends

```
CREATE ROLE my_auditor;
ALTER ROLE ALL SET pgaudit.role = 'my_auditor';
-- Reconnect the session here -- required for the above to take effect.
GRANT SELECT ON t1 TO my_auditor;
```

Confirm `SHOW pgaudit.role`; reflects `my_auditor` only after reconnecting, and confirm a `SELECT` against `t1` now appears in `postgres.log` (its path and name obtainable via `SHOW log_directory`), while a `SELECT` against a different, non-granted table does not.

Section summary In this section, we exercised both halves of this month's article against a real cluster: a PITR-backed single-table recovery via database cloning (rather than a full database rewind), and a correct, working pgAudit configuration sequence that deliberately routes around the two silently-ineffective commands confirmed in section 6.1.5.

6.3 In this document, we reviewed or created

This month and in this document we detailed the following:

- The real distinction between Backups and PITR – different tools for different failure modes, neither a superset of the other, summarized directly against the dimensions that matter in an actual incident (table-level restore, cross-cluster restore, “restore to N seconds ago,” and total cluster loss).
- How a yugabyteDB snapshot actually works under the hood – hard links to existing SST files, not a data copy – and the direct storage-growth consequence of keeping many snapshots or long retention windows as compactions produce new files.

- A real, working pattern for effectively recovering a single table despite PITR itself operating at the database level: `CREATE DATABASE ... TEMPLATE ... AS OF '<timestamp>'`.
- pgAudit's real configuration surface, including several specific, directly-verified gotchas: `SET pgaudit.role` silently does nothing (use `ALTER ROLE/ALTER DATABASE` instead, and reconnect), `ALTER SYSTEM` is not yet supported for pgAudit settings, `TRUNCATE` is not covered by object-level audit scope, and object-level audit scope is controlled by ordinary `GRANT/REVOKE` privileges on the audited table.

Persons who helped this month Kiyu Gabriel, David Bechberger

Additional resources: Free yugabyteDB training courses,

https://university.yugabyte.com/users/sign_in

Take any class, anytime, for free.

Jim Knicely's very excellent blog site,

<https://yugabytedb.tips/>

This document is located here,

<https://www.yugabitten.com/62%20-%20Monthly%20articles/2026-04%20-%20Point-in-Time%20Recovery%20and%20Audit%20Logging/04%20-%20Point-in-time%20recovery%20and%20audit%20logging.pdf>